

MQTT y gestión de suscripciones

Mosquitto · ESP32 · Raspberry Pi / DigitalOcean



Modelo publish/subscribe, topics, QoS, sesiones, retained messages, ACLs y seguridad de broker.

IoT

MQTT 3.1.1 / 5

Mosquitto 2.x

¿Qué problema resuelve MQTT?

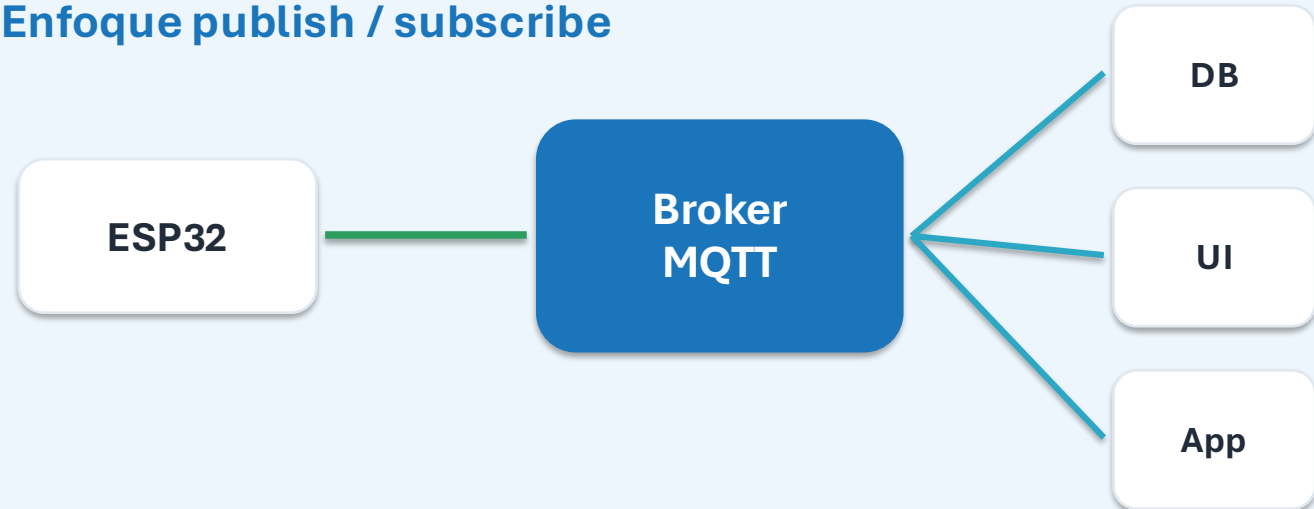
Desacoplar productores y consumidores de datos en sistemas IoT.

Enfoque request/response

El dispositivo debe conocer al servidor, su endpoint y cuándo enviar o consultar información.

Acoplamiento más fuerte entre nodos y aplicaciones.

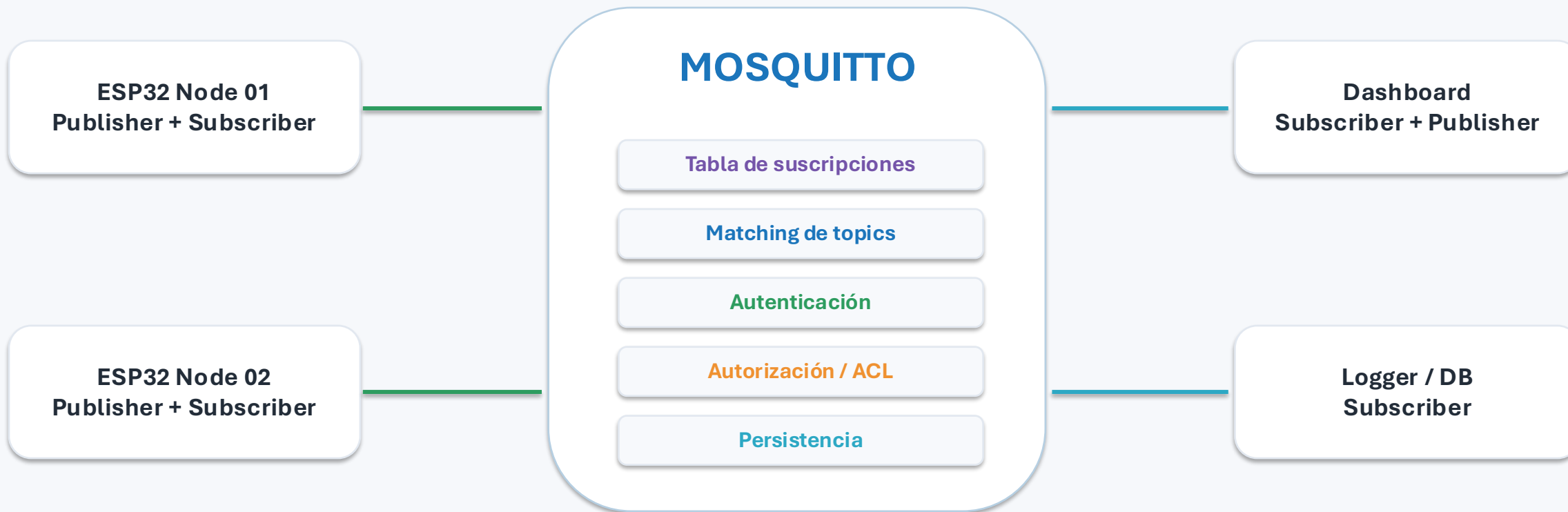
Enfoque publish / subscribe



El ESP32 publica una vez; el broker decide qué clientes deben recibir una copia.

El broker es el punto de desacoplamiento

Mosquitto recibe publicaciones, mantiene suscripciones y aplica políticas.



Un cliente MQTT no queda fijado a un único rol: puede publicar y suscribirse simultáneamente.

Topics y topic filters

Un publisher publica en un topic concreto; un subscriber se registra con un filtro.

PUBLISH → topic + payload

```
iot/node01/sensor/temperature  
23.7
```

Topic concreto

- No usa wildcards.
- Identifica una ruta exacta dentro del árbol de topics.

SUBSCRIBE → topic filter

```
iot/node01/sensor/+  
iot/node01/#
```

Filtro de suscripción

- Puede utilizar + y #.
- Define qué publicaciones desea recibir el cliente.

Wildcards: + y

Permiten suscripciones compactas sobre jerarquías de topics.

+

Exactamente un nivel

```
iot/+/temperature
```

- ✓ iot/node01/temperature
- ✓ iot/node02/temperature
- ✗ iot/node01/sensor/temperature

#

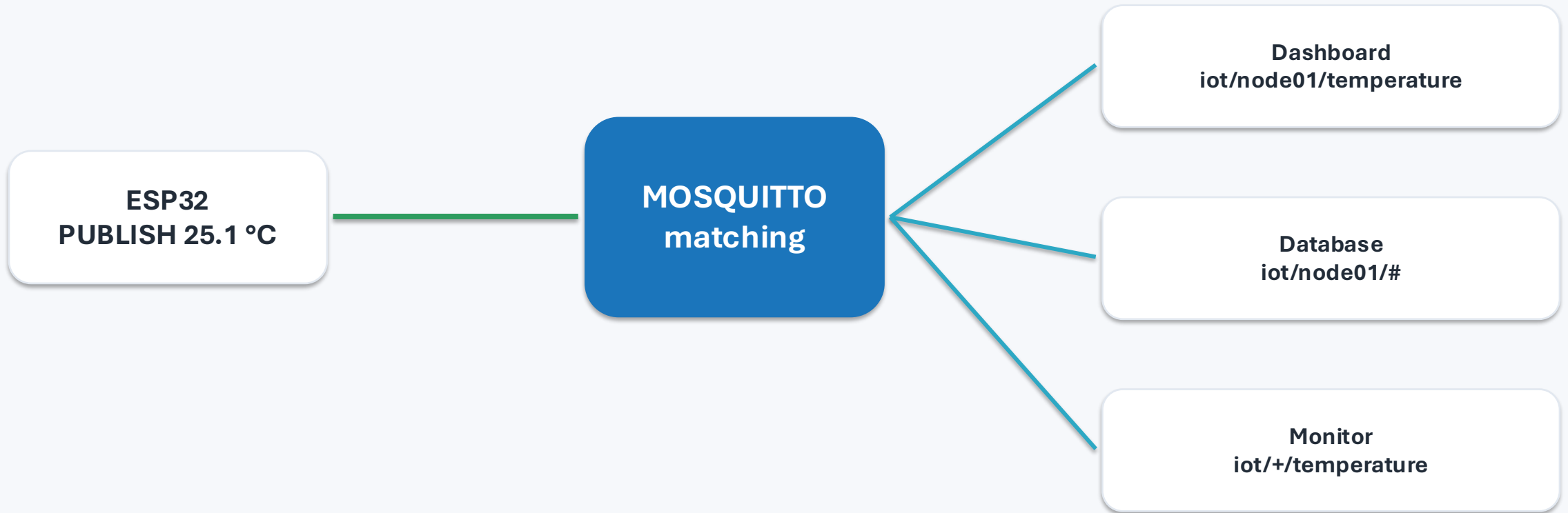
Todos los niveles restantes

```
iot/node01/#
```

- ✓ iot/node01/temperature
- ✓ iot/node01/sensor/temperature
- ✓ iot/node01/command/restart

Una publicación puede llegar a muchos subscribers

El broker entrega una copia a cada suscripción cuyo filtro haga match.



El publisher no necesita conocer cuántos consumidores existen ni dónde están.

ESP32: publicar y suscribirse

Un mismo cliente puede enviar telemetría y recibir comandos.

PubSubClient

```
mqttClient.publish(  
  "iot/node01/sensor/temperature",  
  "23.7"  
);  
  
mqttClient.subscribe(  
  "iot/node01/command/#"  
);  
  
mqttClient.loop();
```

PUBLISH

SUBSCRIBE

Telemetría →
iot/node01/sensor/#

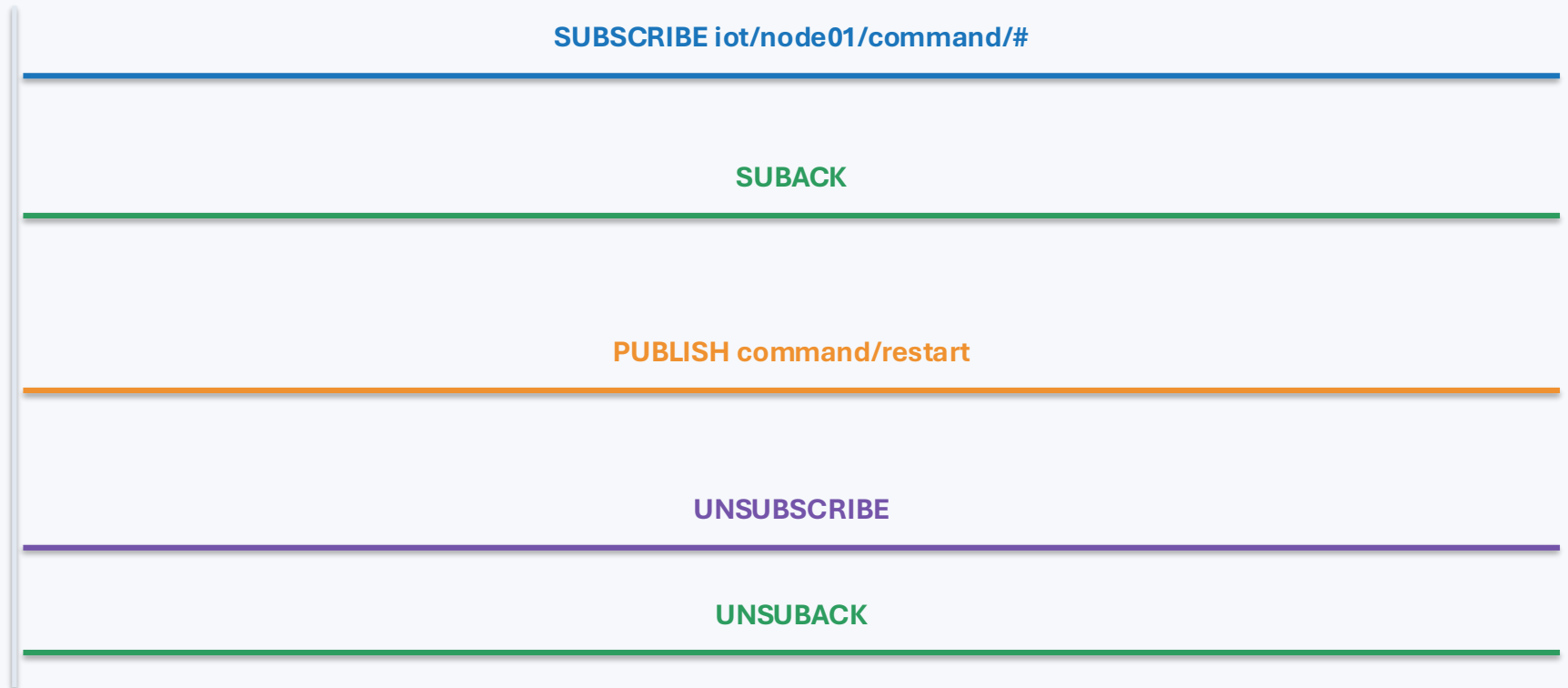
Comandos ←
iot/node01/command/#

Ciclo de vida de una suscripción

La suscripción se crea desde el cliente y el broker la mantiene dentro de la sesión.

Cliente MQTT

Broker



Tres conceptos distintos

No confundir autenticación, autorización y suscripción.

AUTENTICACIÓN

¿Quién eres?

password_file
usuarios / credenciales

AUTORIZACIÓN

¿Qué puedes hacer?

ACL
read / write / topics

SUSCRIPCIÓN

¿Qué quieres recibir?

SUBSCRIBE
topic filter + QoS

ACL: “puedes suscribirte”. SUBSCRIBE: “quiero suscribirme”.

ACLs en Mosquitto

El broker controla qué topics puede leer o escribir cada usuario.

```
/etc/mosquitto/acl
```

```
user esp32_node01
```

```
topic write iot/node01/sensor/#
```

```
topic write iot/node01/status/#
```

```
topic read iot/node01/command/#
```

PERMITIDO

```
iot/node01/sensor/temperature
```

BLOQUEADO

```
iot/node02/sensor/temperature
```

Escalar ACLs con %u y %c

Patrones evitan reglas repetidas para decenas de nodos.

ACL pattern

```
pattern write iot/%u/sensor/#  
pattern read iot/%u/command/#
```

%u → username

%c → client ID

Muy útil para flotas de nodos IoT.

Ejemplo: username = node01

✓ iot/node01/sensor/temperature

✓ iot/node01/command/restart

✗ iot/node02/sensor/temperature

QoS: semántica de entrega

La confiabilidad aumenta, pero también el tráfico y el estado protocolario.

QoS	Semántica	Intercambio	Uso típico
0	At most once	PUBLISH	Telemetría frecuente
1	At least once	PUBLISH + PUBACK	Comandos / eventos
2	Exactly once*	4 pasos	Casos muy específicos

Recomendación práctica: QoS 0 para telemetría periódica; QoS 1 para comandos y alarmas importantes.

*"Exactly once" se refiere a la semántica MQTT entre pares, no a la lógica de negocio extremo a extremo.

Sesiones persistentes

Deciden si el broker conserva estado del cliente entre desconexiones.

Sesión no persistente

- Al desconectarse, se descarta la sesión.
- Las suscripciones desaparecen.
- Al reconectar, el cliente debe suscribirse nuevamente.

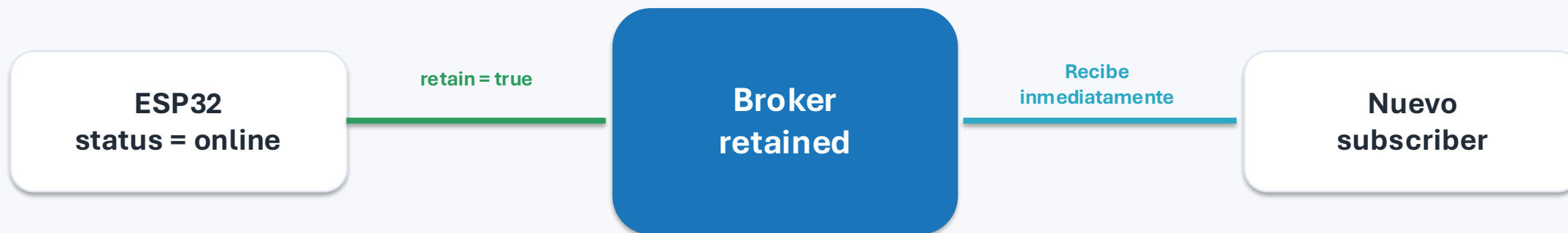
Sesión persistente

- El broker conserva la sesión.
- Puede mantener suscripciones y mensajes aplicables.
- Requiere un Client ID estable.

No confundir “sesión persistente MQTT” con “persistencia en disco del broker”.

Retained messages

El broker conserva el último valor retenido de un topic.



Subscription → recuerda qué quiere un cliente
Retained message → recuerda el último valor de un topic
Persistent session → recuerda estado de la sesión

Monitorear el broker

Mosquitto expone estadísticas y eventos útiles para operación y diagnóstico.

Estadísticas \$SYS

```
mosquitto_sub \  
-h localhost \  
-t "$SYS/#" -v
```

Log del broker

```
tail -f /var/log/mosquitto/mosquitto.log
```

Qué observar

- Clientes conectados
- Mensajes recibidos / enviados
- Bytes / tráfico
- Conexiones y desconexiones
- Errores de autenticación

MQTT no define un comando estándar “listar todas las suscripciones”.

Funciones avanzadas de Mosquitto y MQTT 5

Útiles cuando el número de clientes y servicios crece.

Dynamic Security

- Gestiona clientes, roles, grupos y ACLs en runtime.
- Evita editar manualmente archivos estáticos en sistemas grandes.
- Administración con mosquitto_ctrl.

Shared subscriptions

```
$share/workers/iot/+/sensor/#
```

- Varios workers comparten el mismo filtro.
- Cada mensaje se entrega a un miembro del grupo.
- Útil para balancear procesamiento backend.

Arquitectura recomendada para ESP32 + Raspberry Pi

El broker local actúa como gateway y punto de control del sistema IoT.



**ESP32 publica telemetría y se suscribe a comandos.
El Pi centraliza autenticación, ACLs, monitorización y servicios.**

Jerarquía de topics propuesta

Consistencia de nombres simplifica ACLs, dashboards y automatización.

Topic tree

```
iot/  
├── node01/  
│   ├── sensor/  
│   │   ├── temperature  
│   │   ├── humidity  
│   │   └── pressure  
│   ├── network/rssi  
│   ├── status  
│   └── command/  
│       ├── led  
│       ├── sampling  
│       └── restart  
└── node02/...
```

ESP32 publica

iot/node01/sensor/#
iot/node01/network/#
iot/node01/status

ESP32 se suscribe

iot/node01/command/#

Ideas clave

1

Las suscripciones las crea el cliente; el broker las mantiene y aplica matching.

2

Los ACLs controlan lo permitido, no lo que el cliente “quiere” recibir.

3

QoS, sesiones y retained messages resuelven problemas diferentes.

4

Una jerarquía de topics bien diseñada simplifica seguridad y escalabilidad.

5

ESP32 + Mosquitto en Raspberry Pi es una arquitectura sólida para laboratorios IoT.

Siguiente paso sugerido: laboratorio ESP32 → Mosquitto → dashboard con ACLs y QoS.